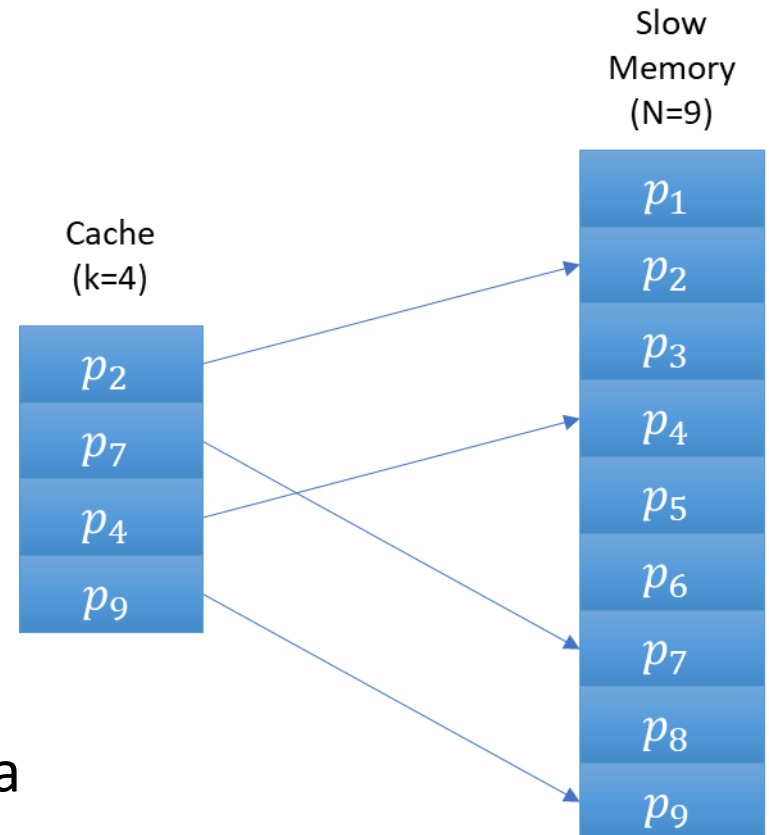


# Deterministic Paging Algorithms

# Paging

- 2-level memory divided into pages of a fixed uniform size
  - $N$  pages in the slow memory
  - $k < N$  pages in the fast memory (“the cache”)
- Given a request for a page  $p_i$ , it should become available in the cache
  - If it is already in the cache, the request costs 0
  - Otherwise, it is a **page fault** - the page is copied from the slow memory to the fast memory with a cost of 1



# The Problem

- Which page should be evicted from the fast memory in the case of a page fault?
- The goal – minimize the total cost == **the number of page faults** for a given access sequence
  - Note: this is different from complexity analysis, as we only count page faults – and not the cost of the implementation
- Typically, only  $k$  is considered a relevant parameter for analysis
  - $N$  considered very large or infinite

# “In the real world”

- Typically, the fast memory is the RAM, and the slow memory is the disk
- Can be applied to general caching problems, usually with a different cost model
  - In a “full access cost model”, fast memory access costs 1 and slow memory access costs some value  $s$
  - Or, equivalently,  $\frac{1}{s}$  and 1

# Some Deterministic Paging Algorithms

- FIFO (First-In, First-Out)
  - Replace the page that's been in the cache longest
- LIFO (Last-In, First-Out)
  - Replace the page most recently moved to the cache
- LRU (Least Recently Used)
  - Replace the page whose most recent access is earliest
- LFU (Least Frequently Used)
  - Replace the page that has been accessed the least
- LFD (Longest Forward Distance)
  - Replace the page whose next access is the latest

# Reminder: Competitive Ratio

- An online algorithm  $ALG$  is called  $c$  – *competitive* if for all finite input sequences  $\sigma$ ,

$$ALG(\sigma) \leq c \cdot OPT(\sigma)$$

- Where  $OPT$  is an **optimal offline algorithm**
- $\mathcal{R}(ALG)$  - The infimum over the set of all values  $c$  such as  $ALG$  is  $c$  – *competitive* is called **the competitive ratio** of  $ALG$

# Weak Competitive Ratio

- An online algorithm  $ALG$  is called *weakly  $c$  – competitive* if there exists a constant  $\alpha$  such that for all finite input sequences  $\sigma$ ,
$$ALG(\sigma) \leq c \cdot OPT(\sigma) + \alpha$$
- Where  $OPT$  is an **optimal offline algorithm**

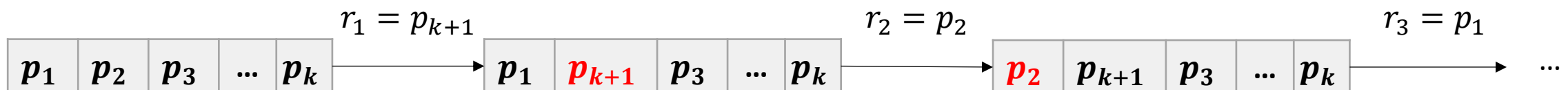
# A Lower Bound

- Let  $A$  be a (competitive) **deterministic** paging algorithm. Then  $\mathcal{R}(ALG) \geq k$ .
  - Note: this lower bound is only for deterministic algorithms, there are better randomized algorithms



# Proof of Lower Bound

- Assume there are  $k + 1$  pages:  $p_1, \dots, p_{k+1}$
- W.L.O.G. assume that initially  $p_1, \dots, p_k$  are in the cache
- Define the cruel adversary recursively as follows:
  - $r_1 = p_{k+1}$
  - $r_{i+1}$  = The unique page not in  $ALG$ 's cache after servicing  $r_1, \dots, r_i$ 
    - $ALG$  is deterministic, so it is well defined
- Since  $ALG$  faults on each request, we get that  $ALG(\sigma) = |\sigma|$



# Proof of Lower Bound (2)

- Now, let's prove that  $LFD(\sigma) \leq \left\lceil \frac{|\sigma|}{k} \right\rceil$ :
  - Suppose that when servicing the  $i$ th request,  $r_i$ ,  $LFD$  evicts the page  $p$
  - According to the definition of  $LFD$ , since there are  $k + 1$  pages, all the other  $k - 1$  pages in the cache (not including  $r_i$ ) must be requested before the next request for  $p$
  - Therefore,  $LFD$  faults at most once every  $k$  requests
- Hence  $LFD(\sigma) \leq \left\lceil \frac{ALG(\sigma)}{k} \right\rceil$ , and  $\mathcal{R}(ALG) \geq k$  ■

# Marking Paging Algorithms

- Let  $\sigma$  be any request sequence. We divide  $\sigma$  into phases as follows:
  - phase 0 is the empty sequence
  - phase  $i$  is the maximal sequence following phase  $i - 1$  that contains at most  $k$  distinct page requests
- For example, with  $k = 3$

|   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 3 | 4 | 3 | 4 | 1 | 2 | 2 | 5 | 1 | 2 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|

# Marking Paging Algorithms

- Let  $\sigma$  be any request sequence. We divide  $\sigma$  into phases as follows:
  - phase 0 is the empty sequence
  - phase  $i$  is the maximal sequence following phase  $i - 1$  that contains at most  $k$  distinct page requests
- For example, with  $k = 3$

|   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 3 | 4 | 3 | 4 | 1 | 2 | 2 | 5 | 1 | 2 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|

# Marking Paging Algorithms

- Let  $\sigma$  be any request sequence. We divide  $\sigma$  into phases as follows:
  - phase 0 is the empty sequence
  - phase  $i$  is the maximal sequence following phase  $i - 1$  that contains at most  $k$  distinct page requests
- For example, with  $k = 3$



# Marking Paging Algorithms

- Let  $\sigma$  be any request sequence. We divide  $\sigma$  into phases as follows:
  - phase 0 is the empty sequence
  - phase  $i$  is the maximal sequence following phase  $i - 1$  that contains at most  $k$  distinct page requests
- For example, with  $k = 3$



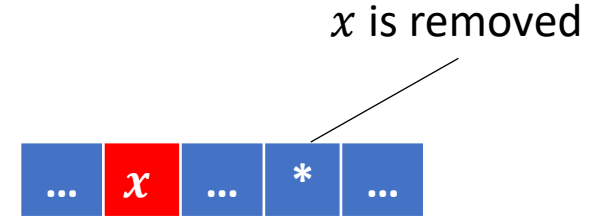
- Such a partition is called a  **$k$ -phase partition**, and it is independent of any specific algorithm

# Marking Paging Algorithms

- Associate with each page of the slow memory a bit called its **mark**
- Suppose that at the beginning of each  $k$ -phase, we unmark all the pages in the cache
- During a  $k$ -phase, we mark a page when it's first requested
- A **marking algorithm** always evicts an unmarked page from its cache

# LRU is a marking algorithm (Proof)

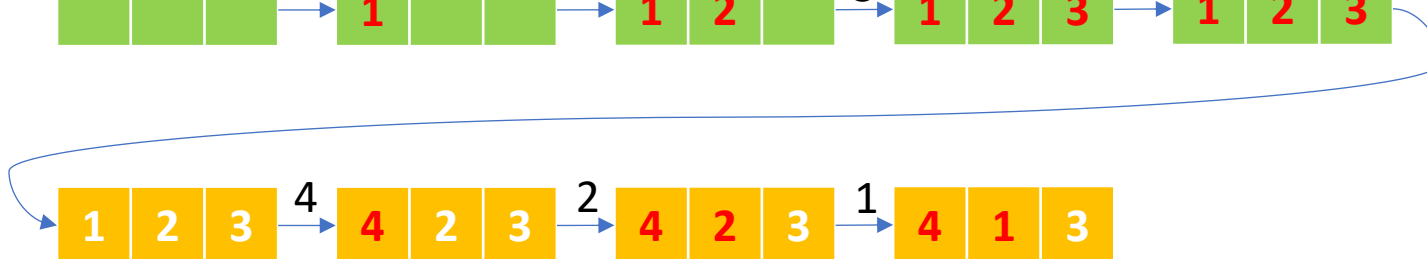
- Suppose by contradiction that *LRU* is not a marking algorithm
- Then it evicts a marked page  $x$  during some  $k$ -phase
  - Meaning there is some request for  $x$  where it is marked



- For  $x$  to leave the cache in *LRU*, it must be the least recently used – meaning at least  $k + 1$  distinct pages should be requested during the  $k$ -phase
  - The request for  $x$
  - The  $k - 1$  distinct page requests after  $x$  is first requested
  - The request that swaps  $x$  out ■



# FIFO is **not** a marking algorithm (example)



# An Upper Bound for Marking Algorithms

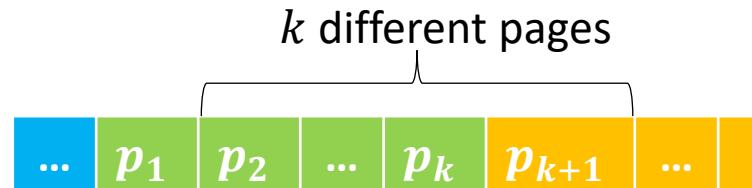
- Let  $ALG$  be any marking algorithm with a cache of size  $k$ . Then  $ALG$  is  $k$  – competitive.

# Proof (1)

- First, show that for any phase  $i \geq 1$ , a marking algorithm *ALG* incurs at most  $k$  page faults:
  - There are  $k$  distinct page references in each  $k$ -phase (except maybe for the last phase)
  - Once a page is accessed, it is marked and can't be evicted until the end of the phase
  - Therefore, *ALG* can't fault twice on the same page – and it faults at most  $k$  times

# Proof (2)

- Second, show that  $OPT$  faults at least once in each phase:
  - Look at the first request in the phase, and the requests until the first request in the next phase



- There are  $k$  pages different from  $p_1$  in that region, and  $p_1$  must be in the cache after the first request – so  $OPT$  must fault at least once
  - $OPT(\sigma) \geq k + l - 2$ , where  $l$  is the number of  $k$ -phases in  $\sigma$ .
- Hence:

$$\frac{ALG(\sigma)}{OPT(\sigma)} \leq \frac{k \cdot l}{k + l - 2} \leq k$$

- Meaning that  $ALG$  is  $k$  – competitive ■

# Corollary

- Every marking algorithm has the optimal competitive ratio possible for a deterministic algorithm
  - *LRU* is “optimal” in that sense

# Conservative Algorithms

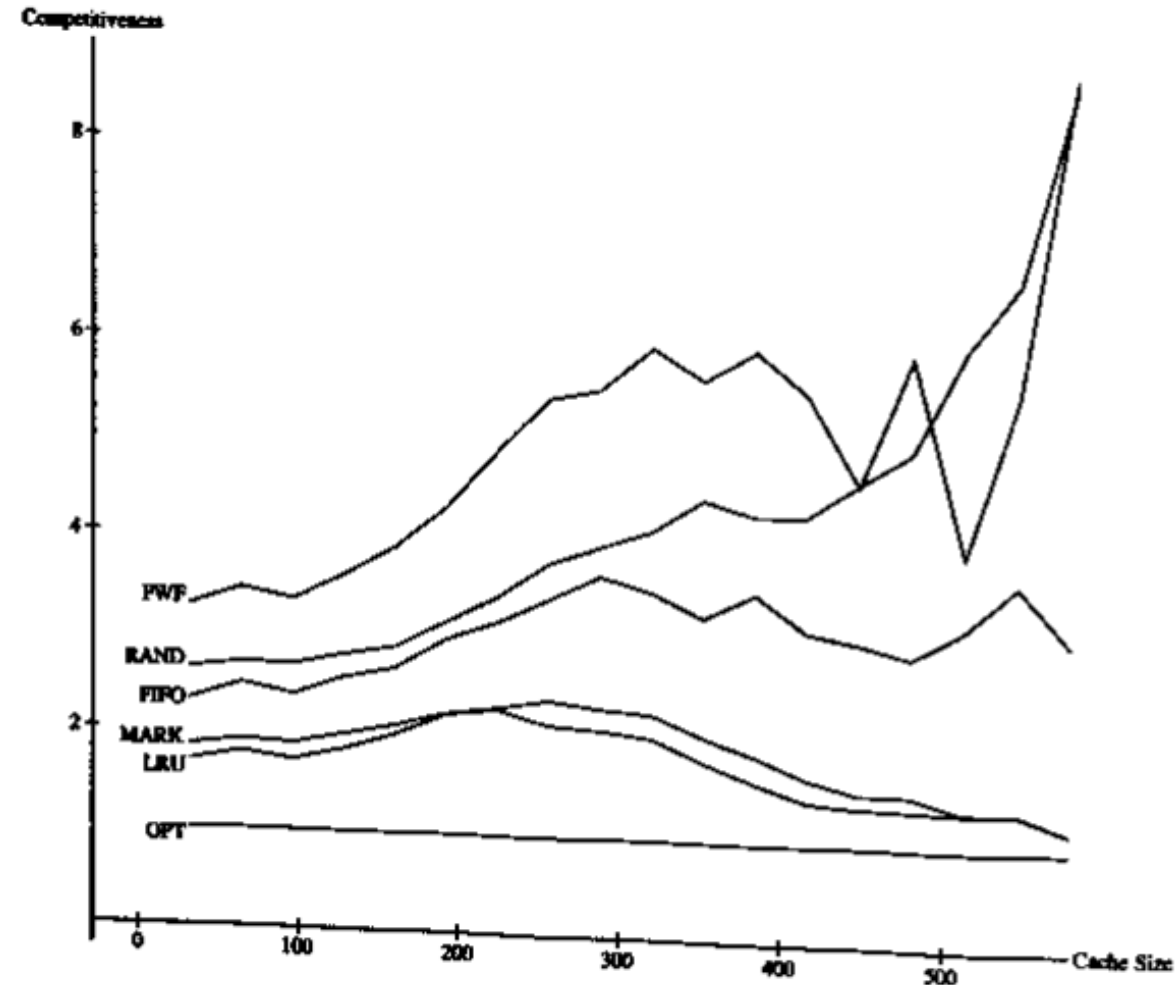
- A paging algorithm is called **conservative** if on any consecutive input subsequence containing at most  $k$  distinct page references, it will incur at most  $k$  page faults
- **Any online conservative paging algorithm is  $k$ -competitive**
  - Proof: same as the proof for marking algorithms, except that now proving that the algorithm incurs at most  $k$  page faults in a  $k$ -phase is immediate

# Conservative vs Marking Algorithms

- LRU is both conservative and marking
- FIFO is conservative
  - → Not every conservative algorithm is marking

# Theory vs Practice

- The previous results suggest that *FWF*, *LRU* and *FIFO* are all “optimal” online solutions
- However, in practice this is not the case
- There are more refined versions of competitive analysis that can show the difference demonstrated by this graph



**Figure 3.1:** Empirical “competitiveness” of various paging algorithms.



# Bibliography

- Allan Borodin and Ran El-Yaniv. 1998. **Online computation and competitive analysis**. Cambridge University Press, USA.
  - Chapters 1+3
- Daniel D. Sleator and Robert E. Tarjan. 1985. **Amortized efficiency of list update and paging rules**. Commun. ACM 28, 2 (Feb. 1985), 202–208. DOI:<https://doi.org/10.1145/2786.2793>